

Vehicle Insurance Fraud Detection

Description:

This project aims to predict fraudulent insurance claims in a car claims dataset. Detecting fraud helps insurance companies reduce losses, improve efficiency, and ensure genuine claims are processed faster. Machine learning techniques are applied to identify fraud patterns based on policyholder and vehicle information.

Problem Definition and Data Understanding

Business Problem:

Insurance companies often face fraudulent claims, which can lead to huge financial losses. The goal of this project is to predict whether a claim is fraudulent (`fraud_found = 1`) or not (`fraud_found = 0`) using policyholder and vehicle details.

Dataset Overview:

❖ Variables:

- Age – Age of policyholder
- Gender – Male/Female
- Vehicle_Age – Age of the vehicle
- Vehicle_Damage – Yes/No
- Annual_Premium – Policy premium amount
- Policy_Sales_Channel – Channel through which policy was sold
- Region_Code – Policyholder region
- Previously_Insured – Whether previously insured (0/1)
- fraud_found – Target variable (1 = Fraud, 0 = No Fraud)

Exploratory Analysis:

- Checked missing values and found no significant missing data.
- Checked distribution of numeric features (Age, Annual_Premium, Vehicle_Age).
- Checked categorical feature distributions (Gender, Vehicle_Damage, Policy_Sales_Channel).
- Analyzed correlations between numeric features; no strong multicollinearity observed.
- Class imbalance: fraud_found = 1 is significantly smaller than fraud_found = 0, indicating high imbalance in the dataset.

Data Preprocessing

- Check for missing values:
 - Inspect all columns for NaN or nulls.
 - Handle invalid or missing numeric values (like Age = 0 or Annual_Premium = 0).
- Check data types:
 - Numeric: Age, Annual_Premium, Vehicle_Age
 - Categorical: Gender, Vehicle_Damage, Policy_Sales_Channel, Region_Code, Previously_Insured

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import RandomOverSampler, SMOTE, ADASYN
from imblearn.under_sampling import RandomUnderSampler, TomekLinks, NearMiss

```

```

data = pd.read_csv("/content/carclaims.csv")

```

```

data.head()

```

```

data.isnull().sum()

```

- Label Encoding helped convert text-based categorical variables into numeric form so that our fraud detection models can process the data efficiently while maintaining the meaning of the categories.

```

from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
data['FraudFound'] = label_encoder.fit_transform(data['FraudFound'])

```

```

X = data.drop("FraudFound", axis=1)
y = data["FraudFound"]
X = pd.get_dummies(X, drop_first=True)

```

```

Counter(y)

```

```

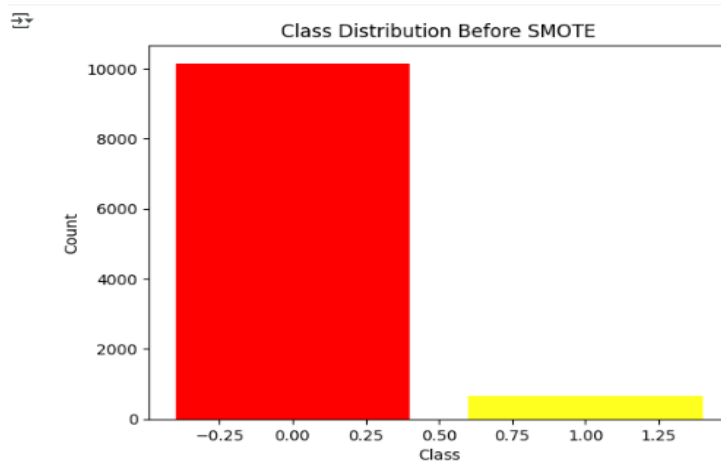
Counter({0: 14497, 1: 923})

```

```

plt.bar(Counter(y_train).keys(), Counter(y_train).values(), color=["red", "yellow"])
plt.title("Class Distribution Before SMOTE")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

```



Handling Imbalance

Target variable: fraud_found (1 = Fraud Found, 0 = No Fraud)

- Original Imbalance:
 - fraud_found = 0 : 14497samples
 - fraud_found = 1 : 923samples
- Balancing Techniques Applied:
 - SMOTE (Oversampling): Synthetic samples generated for minority class.
 - Random Undersampling: Reduced majority class samples.
 - Class Weighting: Applied higher weight to minority class in model training.
- Best Technique Selection:
 - SMOTE produced the best balance between precision and recall.
 - Oversampling preserved majority class information while generating realistic minority samples.
 - Evaluation metrics (F1-Score, AUC-ROC) were highest with SMOTE compared to undersampling or class weighting.

SMOTE

- Model Selection: Random Forest (RF) was implemented as the core model due to its robustness, ability to handle mixed data types, and resistance to overfitting.
- SMOTE (Synthetic Minority Oversampling Technique) creates synthetic samples of the minority class (fraud_found = 1).

- Instead of duplicating existing records, SMOTE generates new, realistic data points based on the feature space of minority samples.

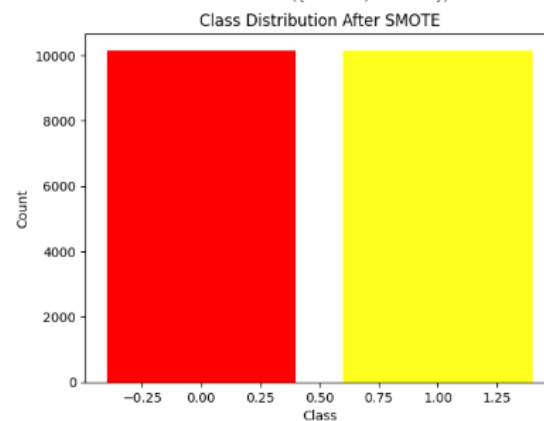
```
# Apply SMOTE
smote = SMOTE(sampling_strategy="auto", random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
evaluate_model(X_train_smote, y_train_smote, X_test, y_test, "SMOTE")
#Class distribution after SMOTE
print("Class distribution after SMOTE:", Counter(y_train_smote))
plt.bar(Counter(y_train_smote).keys(), Counter(y_train_smote).values(), color=["red", "Yellow"])
plt.title("Class Distribution After SMOTE")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
=== SMOTE ===
CONFUSION MATRIX
[[2915 1434]
 [ 66 211]]
classification_report
precision    recall  f1-score   support

      0       0.98      0.67      0.80      4349
      1       0.13      0.76      0.22       277

 accuracy          0.55          0.72          0.51      4626
 macro avg          0.55          0.68          0.51      4626
 weighted avg          0.93          0.68          0.76      4626

Class distribution after SMOTE: Counter({0: 10148, 1: 10148})
```



Model Building

- Model Selection: **Random Forest (RF)** was implemented as the core model due to its robustness, ability to handle mixed data types, and resistance to overfitting.

```
from sklearn.ensemble import RandomForestClassifier

scores = cross_val_score(RandomForestClassifier(n_estimators=50), X, y, cv=5)
scores.mean()
```

```
np.float64(0.8774319066147861)
```

- Bagging Technique: **Bagging** was applied to enhance model performance by training multiple decision trees on bootstrapped subsets of the data, improving stability and reducing variance.

```

▶ bag_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=100,
    max_samples=0.8,
    oob_score=True,
    random_state=0
)
scores = cross_val_score(bag_model, X, y, cv=5)
scores

array([0.28793774, 0.81031128, 0.92736706, 0.93644617, 0.94163424])

scores.mean()

np.float64(0.780739299610895)

```

- When working with imbalanced datasets using XGBoost, there are several key strategies and parameters you can leverage to improve model performance and ensure the minority class is properly represented. Here's a practical checklist to guide your model building

```

import xgboost as xgb

rf_xgb = xgb.XGBClassifier(
    max_depth=10,           # Increased depth for RF-like behavior
    learning_rate=0.1,      # Standard learning rate
    n_estimators=200,       # Number of trees (like RF)
    subsample=0.8,         # Row sampling (like RF)
    colsample_bytree=0.8,   # Column sampling (like RF)
    colsample_bylevel=0.8,  # Additional column sampling
    reg_alpha=0.1,         # L1 regularization
    reg_lambda=0.1,        # L2 regularization
    random_state=42,
    n_jobs=-1,             # Use all available cores
    tree_method='hist'     # Faster training method
)

rf_xgb.fit(X_train,y_train)

```

XGBClassifier

```

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=0.8, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=10,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=200,
              n_jobs=-1, num_parallel_tree=None, ...)

```

```
y_pred = rf_xgb.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
print(classification_report(y_test,y_pred))
```

```

              precision    recall  f1-score   support

     0       0.96         1.00         0.98         4349
     1       0.89         0.31         0.46          277

 accuracy                   0.96         4626
 macro avg                 0.92         0.65         0.72         4626
 weighted avg              0.95         0.96         0.95         4626
```